1. Node Declaration DSL

Declare custom Godot nodes with the two-tier macro DSL:

```
require "lapis"

node Player < CharacterBody3D do
  prop speed : Float32 = 12.0
  prop jump_velocity : Float32 = 4.5
  signal health_changed(new_hp : Int32)
  signal died

  on_ready do
    puts "Player initialized in scene tree"
  end
end
```

- Subclasses Godot engine classes (`Node2D`, `Node3D`, `CharacterBody3D`, `Control`).
- Statically registered with Godot GDExtension at boot.

2. Properties & Exports

Expose typed properties to the Godot Inspector:

```
# Basic primitive types
prop max_hp : Int32 = 100
prop gravity : Float32 = 9.8
prop is_alive : Bool = true
prop character_name : String = "Hero"

# Vectors & Engine types
prop spawn_point : Vector3 = Vector3.ZERO
prop tint : Color = Color.WHITE

# Resource export
prop texture : Texture2D?
```

- Zero runtime reflection; getters/setters synthesized at compile time.

3. Signals & Event Binding

Define and emit custom signals:

```
# Signal definition
signal score_awarded(points : Int32)
signal checkpoint_reached(id : String)

# Emitting signals
emit_signal(:score_awarded, 100)
emit_signal(:checkpoint_reached, "zone_1")

# Connecting signals
target.connect("score_awarded") do |pts|
  handle_score(pts.as_i64.to_i32)
end
```

4. Frame & Physics Lifecycle

Hook into engine loop ticks:

```
# Idle/Render frame process (variable delta)
on_process do |delta|
  rotate_y(Float32.new(delta * 1.5))
end

# Physics process (fixed tick rate, default 60Hz)
on_physics_process do |delta|
  handle_movement(delta)
end

# Tree notification hooks
on_enter_tree { setup_listeners }
on_exit_tree  { teardown_listeners }
```

5. CharacterBody3D & Movement

Idiomatic 3D character controller movement:

```
def handle_movement(delta : Float64)
  dir = Input.get_vector(
    "ui_left", "ui_right",
    "ui_up", "ui_down"
  )

  vel = velocity
  vel.x = dir.x * speed
  vel.z = dir.y * speed

  unless is_on_floor
    vel.y -= gravity * Float32.new(delta)
  end

  self.velocity = vel
  move_and_slide
end
```

6. Scene Tree & Node Queries

Traversal and node manipulation:

```
# Fetch relative node with strong casting
camera = get_node("Camera3D").as(Camera3D)
hud = find_child("HUD", recursive: true)

# Adding and instantiating children
bullet = bullet_scene.instantiate.as(Node3D)
get_parent.add_child(bullet)
bullet.global_position = muzzle.global_position

# Safe destruction
queue_free
```

7. Vectors & Math DSL

High-throughput vector operations:

```
v1 = Vector3.new(1.0, 2.0, 3.0)
v2 = Vector3.FORWARD

dist = v1.distance_to(v2)
dir  = (v2 - v1).normalized
dot  = v1.dot(v2)
cross = v1.cross(v2)
blend = v1.lerp(v2, 0.5)

# 2D Vectors
pos2d = Vector2.new(100.0, 200.0)
angle = pos2d.angle
```

8. Input Handling & Actions

Query configured Godot `InputMap`:

```
# Digital / Boolean button state
if Input.is_action_just_pressed("jump")
  jump
end

if Input.is_action_pressed("sprint")
  apply_boost
end

# Analog axes (gamepad sticks / WASD)
dir = Input.get_vector("left", "right", "up", "down")
axis = Input.get_axis("brake", "throttle")
```

9. Two-Tier Engine Codex Axioms

- **Tier 1 (Engine)**: Pure algorithms, zero macro bloat, no dynamic reflection, high throughput.
- **Tier 2 (DSL)**: Fluent builders, compile-time AST macros, declarative syntax.
- **Hot Path Rule**: Zero heap allocations inside `on_process` or `on_physics_process`.
- **Target Rule**: Lapis PRs target Godot branch `4.8-dev7`, never `main`/`master`.